Arbor

Version Control System

Technical Documentation

Version 1.1

Date of Publication

April 2026

Developed by

Technical University of Sofia and SAP Labs

@2026

CONTENTS

INTRODUCTION 1

1. **SECTION 1 – SYSTEM ARCHITECTURE** 2
2. **SECTION 2 – DATA TRANSFER OBJECTS (DTOs)** . 3
   - 2.1 – Authentication DTOs 3
   - 2.2 – Document DTOs 4

3. **SECTION 3 – DATA MODELS** 6
   - 3.1 – User 6
   - 3.2 – Document 6
   - 3.3 – Document Version 6
   - 3.4 – Document Status (Enum) 7
   - 3.5 – Role Request 7

4. **SECTION 4 – CORE SERVICES** 8
   - 4.1 – Auth Service 8
   - 4.2 – Document Service 9
   - 4.3 – Role Service 10

5. **SECTION 5 – SECURITY** 11

6. **SECTION 6 – API ENDPOINTS** 12

7. **SECTION 7 – MIDDLEWARE** 13

8. **SECTION 8 – DATABASE LAYER** 14
   - 8.1 – Uses 14
   - 8.2 – Key Tables 14
   - 8.3 – Entity Relationship Diagram 15

9. **SECTION 9 – TRANSACTION MANAGEMENT** . . . 17

10. **SECTION 10 – EXCEPTION HANDLING AND API** . . 18

11. **SECTION 11 - AUDIT TRAIL** 20

12. **SECTION 12 - ENVIRONMENT VARIABLES** . . . 21

Introduction

Welcome to the technical documentation for Arbor Version Control System.

In modern software environments, managing document history and ensuring proper oversight is critical. Arbor addresses these needs by providing a lightweight yet powerful platform built on Java and MySQL. The system's core philosophy centers on Transparency, Security, and Traceability.

1.1 Key Capabilities:

- Version Control: Automatic versioning and historical tracking of every document change.
- Approval Workflow: A built-in lifecycle (Pending → Approved/Rejected) for quality assurance.
- Role-Based Security: Granular access control using JWT authentication to protect sensitive data.
- Audit Logging: Comprehensive tracking of all administrative and document-related actions

1.2 Purpose of the Document

This documentation serves as a comprehensive technical guide for developers, system administrators, and stakeholders. It outlines the system architecture, data models, API endpoints, and security protocols implemented within the Arbor ecosystem.

1.3 Target Audience

- **Backend Developers:** To understand the service logic and database interactions.
- **Frontend Developers:** To integrate with the RESTful API and handle data transfer objects (DTOs).
- **QA Engineers:** To verify business rules, role-based access control (RBAC), and error handling.

Section 1

System Architecture

The system follows a **client-server architecture** implemented using a lightweight REST server.

- The backend is implemented in **Java** using the Javalin framework;
- The system communicates via HTTP using JSON format;
- A **MySQL database** is used for persistent storage;
- Authentication is handled via **JWT (JSON Web Tokens).**

Architecture Flow

Client → HTTP Request → Javalin Server → Services → Database (MySQL)

Security & JWT

Authentication & Authorization The system uses **JWT (JSON Web Tokens)** for securing API endpoints.

- **Algorithm:** HMAC256.
- **Token Lifetime:** 20 hours (72,000,000 ms).
- **Security Filter:** A global interceptor (app.before) validates the Authorization: Bearer <token> header for all /api/* routes, except for login and registration.
- **Context Injection:** Upon successful validation, the User object is injected into the request context, making the current user's ID and Role available to the service layer.

Section 2

Data Transfer Objects (DTOs)

DTOs are used to transfer data between the client and the server.

They separate external API structure from internal models.

2.1 Authentication DTOs

2.1.1 LoginRequest

Used when a user logs in.

Fields:

- ❖ username
- ❖ password

2.1.2 LoginResponse

Returned after successful login.

Fields:

- ❖ token (JWT)
- ❖ username
- ❖ role

2.1.3 RegisterRequest

Fields:

- ❖ username
- ❖ password
- ❖ role

2.1.4 ChangePasswordRequest

Fields:

- username
- oldPassword
- newPassword

2.2 Document DTOs

2.2.1 CreateDocumentRequest

Fields:

- title
- content

2.2.2 CreateVersionRequest

Fields:

- content

2.2.3 DocumentDTO

Used when returning document summaries.

Fields:

- id
- title
- author
- activeVersionNumber
- status

2.2.4 PendingVersionDTO

Used for reviewers.

Fields:

- documentId
- documentTitle
- versionNumber
- author

- content
- createdAt

2.2.5 ReviewVersionRequest

Fields:

- status (APPROVED / REJECTED)

Section 3

Data Models

These represent the **core domain objects** in the system.

3.1 User

Represents an authenticated system user.

Fields:

- id
- username
- role (AUTHOR, REVIEWER, READER, ADMIN)

3.2 Document

Represents a document entity.

Fields:

- id
- title
- createdBy
- createdAt
- history (List of versions)

3.3 Document Version

Represents a specific version of a document.

Fields:

- id
- documentId
- versionNumber
- content
- status (PENDING_APPROVAL, APPROVED, REJECTED)
- createdBy
- createdAt

3.4 Document Status (Enum)

Defines possible version states:

- ✓ PENDING_APPROVAL
- ✓ APPROVED
- ✓ REJECTED

3.5 Role Request

Represents a request for role change.

Fields:

- id
- username
- currentRole
- requestedRole
- status
- createdAt

Section 4

Core Services

4.1 Auth Service

Handles authentication and user management.

Responsibilities:

- User registration
- User login
- Password change

Key Methods

- **register()**
  - Hashes password using BCrypt
  - Inserts user into database
  - Returns success status

- **login()**
  - Retrieves user from DB
  - Verifies password
  - Returns User object

- **changePassword()**
  - Verifies old password
  - Hashes new password
  - Updates database

4.2 Document Service (Core Business Logic)

This is the **most important service in the system**.

createDocument()

- Only AUTHOR or ADMIN allowed
- Creates document
- Creates version 1 automatically
- Logs action in audit_logs
- Uses transaction (commit/rollback)

createNewVersion()

- Gets latest version number
- Creates new version (PENDING_APPROVAL)
- Logs action
- Uses transaction

reviewVersion()

- Only REVIEWER or ADMIN allowed
- Changes version status (APPROVED / REJECTED)
- Logs action

getDocumentWithHistory()

- Returns document + all versions
- Versions sorted descending

getDocumentList()

- Returns only ACTIVE documents (if no filter)
- Can filter by author

getPendingVersions()

- Returns versions waiting for approval

addComment()

- Adds reviewer comment to a version

4.3 Role Service

Handles role management.

Features:

- Request role change
- View pending requests
- Approve/reject requests

Special Logic:

- If request is APPROVED:
  → updates user role automatically

Section 5

Security

5.1 Password Security

Uses BCrypt:

- One-way hashing
- Salt included
- Secure password storage

5.2 JWT Authentication

Handled via JWT.

Token contains:

- user ID
- username
- role
- expiration time

Flow:

1. User logs in
2. Token is generated
3. Client sends token in header:
    - Authorization: Bearer <token>
4. Server validates token before each request

Section 6

API Endpoints

6.1 API Table – Version 1 - JPEG

Method	Endpoint	Description	Auth Required
POST	/api/register	Registers a new user with a specific role.	No
POST	/api/login	Authenticates user and returns a JWT.	No
PUT	/api/change-password	Updates the password after verifying the old one.	Yes
POST	/api/documents	Creates a new document and its initial version.	Yes (Author/Admin)
POST	/api/documents/{id}/versions	Submits a new version for approval.	Yes (Author/Admin)
PUT	/api/documents/{id}/review	Approves or rejects a pending version.	Yes (Reviewer/Admin)
GET	/api/documents/pending	Retrieves all versions awaiting review.	Yes (Reviewer/Admin)

6.2 API Table – Version 2 – Regular Table

Method	Endpoint	Description	Auth Required
POST	/api/register	Registers a new user with a specific role.	No
POST	/api/login	Authenticates user and returns a JWT.	No
PUT	/api/change-password	Updates the password after verifying the old one.	Yes
POST	/api/documents	Creates a new document and its initial version.	Yes (Author/Admin)
POST	/api/documents/{id}/versions	Submits a new version for approval.	Yes (Author/Admin)
PUT	/api/documents/{id}/review	Approves or rejects a pending version.	Yes (Reviewer/Admin)
GET	/api/documents/pending	Retrieves all versions awaiting review.	Yes (Reviewer/Admin)

Section 7

Middleware (Security Filter)

Implemented in MainServer.

Function:

- Intercepts all /api/* requests
- Skips login/register
- Validates JWT token
- Extracts user and attaches to request context

Section 8

Database Layer

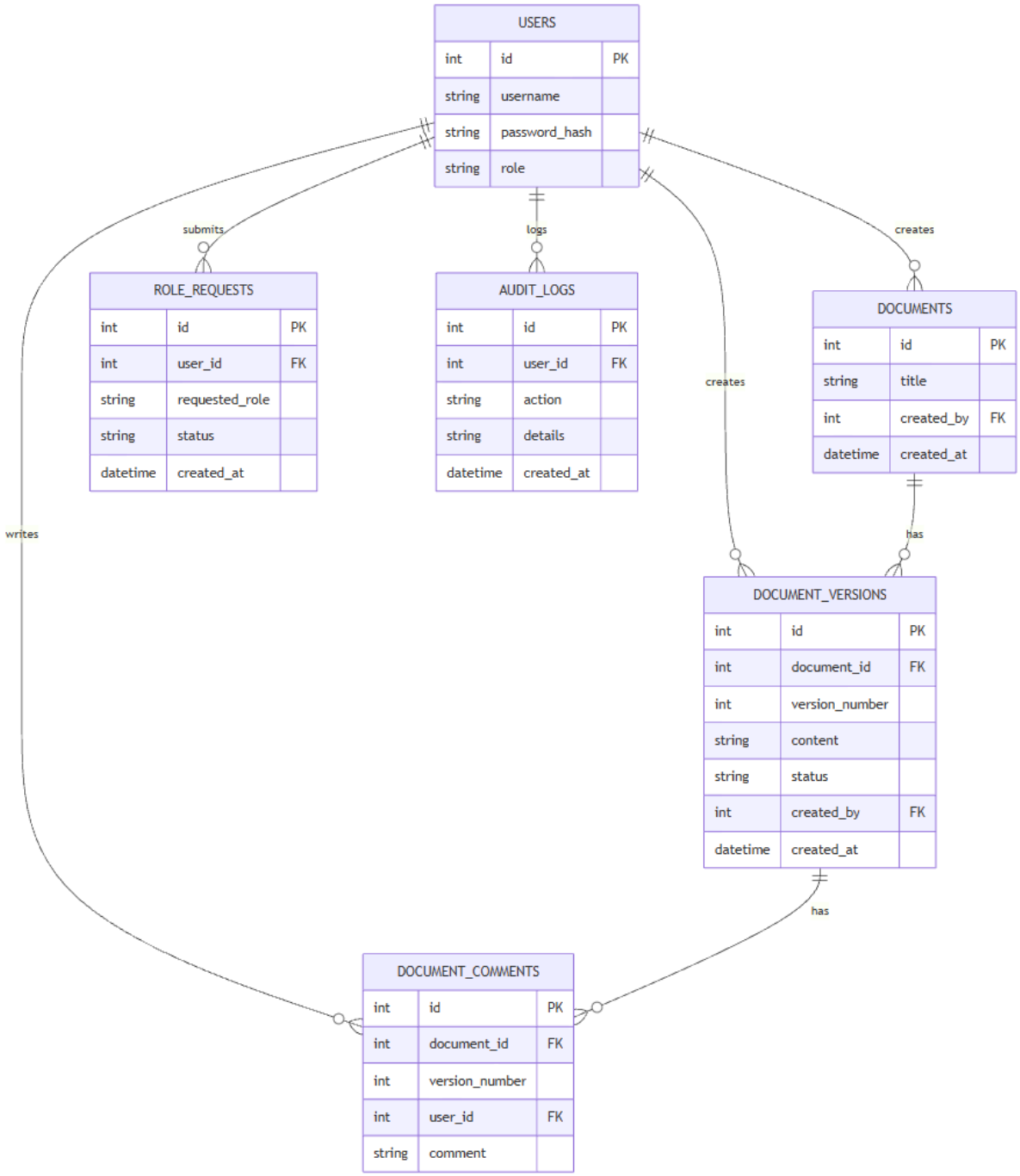
8.1 Uses:

- JDBC (Connection, PreparedStatement)
- Manual SQL queries

8.2 Key Tables:

- users
- documents
- document_versions
- document_comments
- role_requests
- audit_logs

8.3 Entity Relationship Diagram



The system database is designed using a relational model.

Key Relationships:

- A User can create multiple Documents.
- A Document can have multiple Versions.
- Each Version belongs to exactly one Document.

- A User can create multiple Versions.
- Comments are associated with specific document versions.
- Role Requests are submitted by users.
- Audit Logs track all important actions performed by users.

Section 9

Transaction Management

Used in critical operations:

- createDocument
- createNewVersion
- reviewVersion
- role handling

9.1 Mechanism:

connection.setAutoCommit(false)

→ execute queries

→ commit OR rollback

9.2 Atomic Operations

Critical operations such as createDocument and createNewVersion are wrapped in **manual SQL transactions** to ensure consistency:

1. connection.setAutoCommit(false) is invoked at the start.
2. The system performs multiple inserts (e.g., creating a Document, then its first Version, then an Audit Log entry).
3. If all steps succeed, connection.commit() is called.
4. In case of any SQLException or business logic error, connection.rollback() is triggered to prevent partial data entry.

Section 10

Exception Handling and API

The system implements a standardized error reporting mechanism to ensure predictable communication between the server and the client. All errors are returned in JSON format with a clear descriptive message.

10.1 Error Response Format

Whenever an operation fails, the server responds with the following JSON structure:

```
{
  "error": "Detailed description of the error"
}
```

10.2 HTTP Status Code Mapping

The backend maps specific failure scenarios to standard HTTP status codes as follows:

Status Code	Description	Example Scenarios
400 Bad Request	Input validation failed or business logic violation.	Empty document title, duplicate username during registration, or invalid status transition.
401 Unauthorized	Missing or invalid authentication credentials.	Invalid login credentials, expired JWT token, or missing Authorization header.
403 Forbidden	The user is authenticated but lacks the required permissions.	A READER attempting to create a document or a REVIEWER attempting to change their own role.
500 Internal Error	An unexpected server-side or database failure.	Database connection loss or unhandled SQL exceptions.

Status Code	Description	Example Scenarios
400 Bad Request	Input validation failed or business logic violation.	Empty document title, duplicate username during registration, or invalid status transition.
401 Unauthorized	Missing or invalid authentication credentials.	Invalid login credentials, expired JWT token, or missing Authorization header.
403 Forbidden	The user is authenticated but lacks the required permissions.	A READER attempting to create a document or a REVIEWER attempting to change their own role.
500 Internal Error	An unexpected server-side or database failure.	Database connection loss or unhandled SQL exceptions.

10.3 Validation Logic (Service Layer)

Errors are caught and handled primarily in the Service layer, which enforces the following rules:

- Null/Empty Checks: Requests containing null or empty fields for critical data (Title, Content, Password) trigger a 400 Bad Request.
- Database Constraints: Any violation of database integrity (e.g., unique constraints) is caught and returned as a descriptive error message.
- Transaction Rollback: In case of an error during multi-step operations (e.g., createDocument), the system performs a full rollback to maintain data consistency.

Section 11

Audit Trail

The audit_logs table records the following system actions:

- CREATE_DOCUMENT: Logged when a new document is successfully initialized.
- CREATE_VERSION: Logged when a new version is submitted.
- REVIEW_VERSION: Logged when a reviewer approves or rejects a version.
- Details: Each log entry includes the user_id, a timestamp, and specific metadata (e.g., Document ID and status change).

Section 12

Environment Variables

Environment variables:

- JWT_SECRET: The secret key used for signing tokens. (Default fallback provided for development).
- DB_URL: jdbc:mysql://localhost:3306/project
- DB_USER: Database username.

@2026